OAT | PARTE COMUM E PARTE DE SEGURANÇA

LoginGuard

Gestão de logins, contas e acessos pelo terminal

Grupo: LoginGuard
Disciplina: Segurança e Otimização de Banco de Dados
Documento: entrega inicial de arquitetura e planejamento

Nome completo	Matrícula
Luid Moura Santana	262070177
Alex Gabriel Alves Dos Santos	241070261
Filipe Alves Ferreira Rocha	222700092
Gustavo Muniz dos Anjos	241070612

Repositório do projeto: github.com/Guztaver/loginguard-bd

Problema, público e resultado esperado

Equipes de suporte precisam cadastrar contas, controlar bloqueios e investigar acessos sem expor credenciais ou permitir alterações por pessoas sem autorização. Quando o projeto funcionar, cada operador terá acesso apenas às operações e aos dados necessários ao seu trabalho.

Escopo da versão mínima

Programa de terminal em Python conectado ao SQL Server, com cadastro, autenticação, bloqueio e desbloqueio, três perfis de operação e histórico de tentativas. O projeto gerencia contas de usuários do sistema. As identidades que se conectam ao banco formam uma camada separada de autorização.

O recorte corresponde à Parte comum e à Parte de Segurança. A interface será terminal e os dados operacionais serão totalmente fictícios. O MVP será desenvolvido em três semanas, conforme o canvas.

Estado real da entrega

A proposta e os cinco scripts SQL estão no repositório. O aplicativo e a autenticação ainda serão implementados. A execução dos scripts e a carga do banco não foram comprovadas: o SQL Express local permanece parado. Os planos de teste deste documento não representam resultados já obtidos.

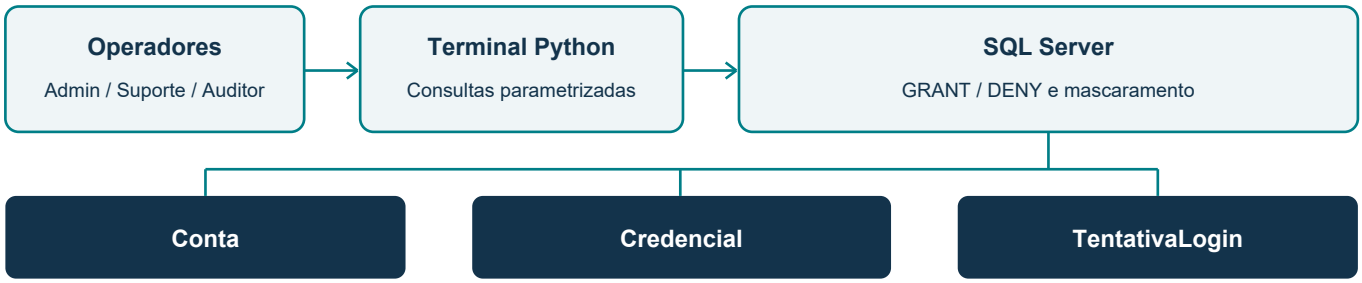
O repositório está privado. O professor precisa ter acesso para avaliar o código pelo link. Os scripts de criação/carga e de permissões também estão reproduzidos nos apêndices deste PDF.

PARTE COMUM | CANVAS DE UMA PÁGINA

Canvas do projeto

Elemento	Definição
Problema e usuário	Contas sem controle de bloqueio e permissões excessivas. Usuários: administrador, suporte e auditor.
Entrada	Nome e e-mail fictícios, identificador de conta, senha digitada de forma oculta e comandos no terminal.
Saída	Resultado genérico da autenticação, confirmação de operações, situação das contas e histórico de tentativas.
Maior risco	Leitura indevida de hashes ou alteração de contas por usuário sem permissão.
Proteções	Papéis mínimos, DENY explícito, credenciais separadas, hash Argon2id e mascaramento de e-mail/IP.
MVP em três semanas	Cadastro, login, bloqueio/desbloqueio, três perfis, histórico e duas consultas otimizadas.
Dados e escala	Carga inicial prevista: 50 contas, 50 registros de credencial sem senha e 500 eventos fictícios. Experimento: mais 200 mil eventos.
Como provar	Testes por papel e comparação de leituras lógicas, tempo e plano antes/depois dos índices.

Arquitetura em caixas



O terminal, a autenticação e o acesso restrito do serviço são componentes planejados. Os scripts já descrevem as tabelas e as permissões dos três papéis humanos.

Semana 1: banco, carga e permissões. **Semana 2:** terminal e autenticação. **Semana 3:** testes, índices e evidências.

Campos e proteção

Banco escolhido: SQL Server 2016 ou superior. Não há campos públicos: os metadados são internos e os dados de contato/credenciais são sensíveis.

Tabela	Campo	Classe	Proteção prevista
Conta	ContaId	Interno	PK; acesso por papel.
Conta	Nome	Interno	Fictício; leitura limitada aos operadores.
Conta	Email	Sensível	Máscara email(); DENY UNMASK.
Conta	Bloqueado	Interno	Alteração apenas pelo administrador.
Conta	CriadoEm	Interno	Leitura por papel; padrão UTC.
Credencial	ContaId	Interno	PK e FK; tabela separada e restrita.
Credencial	HashSenha	Sensível	Argon2id na aplicação; DENY SELECT aos três papéis humanos.
Credencial	AtualizadoEm	Interno	Restrito junto às credenciais.
TentativaLogin	TentativaId	Interno	PK; histórico somente leitura para operadores autorizados.
TentativaLogin	ContaId	Interno	FK opcional; não copia nome ou e-mail.
TentativaLogin	OcorridoEm	Interno	Filtro temporal; histórico protegido.
TentativaLogin	Sucesso	Interno	Resultado da tentativa; acesso por papel.
TentativaLogin	IpOrigem	Sensível	Máscara default(); omitido da view de histórico.

Relacionamentos e dados iniciais

Conta (1) para Credencial (0..1); Conta (1) para TentativaLogin (0..N). Uma tentativa para conta inexistente pode usar ContaId = NULL.

A carga usa nomes "Pessoa Fictícia N", e-mails @example.invalid e IPs 192.0.2.0/24. Os eventos são sintéticos. Todas as contas começam bloqueadas e com HashSenha = NULL; não são contas utilizáveis.

O Argon2id será aplicado por biblioteca na aplicação, com salt e parâmetros incluídos no hash. O CHECK do banco valida apenas o prefixo. Mascaramento não substitui autorização nem elimina inferências; a conexão futura deverá usar TLS com certificado validado.

Quem pode fazer o quê

Operação	Administrador	Suporte	Auditor
Consultar contas com e-mail mascarado	Sim	Sim	Sim
Cadastrar conta e alterar nome, e-mail ou bloqueio	Sim	Não	Não
Inserir/atualizar hash para cadastro ou redefinição	Sim[1]	Não	Não
Ler hashes de senha	DENY	DENY	DENY
Consultar histórico sem IP	Sim	Não	Sim
Inserir, alterar ou excluir histórico diretamente	DENY	DENY	DENY
Alterar papéis ou estrutura do banco	Não concedido	Não concedido	Não concedido

[1] Por componente confiável que gera o hash. O administrador funcional pode substituir hashes, mas não ler seu conteúdo. Identidades de teste: demo_admin, demo_suporte e demo_auditor, todas WITHOUT LOGIN.

Os DENYs aparecem em Credencial (SELECT) e TentativaLogin (INSERT, UPDATE, DELETE). Suporte e auditor também têm escrita negada em Conta e Credencial. Nenhum dos três papéis recebe UNMASK.

Ameaça principal e como demonstrar a correção

Escalonamento de privilégio: o suporte tenta desbloquear uma conta, ler hashes ou se adicionar ao papel de administrador. O auditor tenta apagar o histórico.

Teste previsto	Resultado esperado
Suporte: SELECT em Credencial	Erro de permissão.
Suporte: UPDATE em Conta	Erro de permissão; conta preservada.
Suporte: ALTER ROLE de administrador	Erro de permissão; papel preservado.
Auditor: DELETE em TentativaLogin	Erro de permissão; histórico preservado.
Consulta autorizada pela view	Leitura funciona; e-mail aparece mascarado.

Executar sql/03_testes_permissoes.sql com EXECUTE AS USER e guardar a saída. Critério de aprovação: todas as operações proibidas são negadas e as permitidas funcionam. Testes ainda não executados.

Teste adicional planejado: usar uma entrada de SQL Injection no terminal e verificar que ela é tratada como dado por consultas parametrizadas. O serviço de autenticação terá identidade restrita própria. Não usar db_owner/sysadmin; donos dos objetos e sysadmin não ficam limitados por esses DENYs.

Duas consultas sob volume

Consulta candidata a gargalo	Índice proposto
Q1. Últimas 50 tentativas de uma conta em um período.	(ContaId, OcorridoEm DESC) INCLUDE (Sucesso)
Q2. Dez contas com mais falhas em um período.	Filtrado: (OcorridoEm, ContaId) WHERE Sucesso = 0

Geração de volume

O script 01 prevê 50 contas, 50 registros de credencial e 500 tentativas. O script 04 acrescenta 200 mil eventos por geração numérica, distribuídos entre contas e horários. A quantidade pode ser ajustada na variável @quantidade. Não há resultados reais de autenticação na massa.

Medição antes e depois

1. Executar a criação/carga e a carga adicional antes dos índices. Habilitar o plano de execução real no SSMS e SET STATISTICS IO, TIME ON.
2. Executar uma rodada de aquecimento e cinco medições das duas consultas.
3. Aplicar sql/05_indices.sql e repetir apenas as consultas com o mesmo volume e parâmetros.
4. Comparar a mediana do tempo, leituras lógicas e plano de execução. Confirmar resultados equivalentes.

Não limpar cache global de instância compartilhada. Considerar o efeito de aquecimento e de estatísticas. Medir também o custo dos índices nas escritas e no armazenamento. As consultas são candidatas a gargalo; o experimento deverá comprovar se são lentas no equipamento.

Evidência a coletar	Antes	Depois
Q1: leituras lógicas / mediana ms / plano	Pendente	Pendente
Q2: leituras lógicas / mediana ms / plano	Pendente	Pendente
Linhas por tabela e custo de carga	Pendente	Pendente

Critério e cronograma

Meta: reduzir leituras lógicas sem alterar o resultado das consultas. Não declarar ganho percentual antes da medição.

Semana	Resultado mínimo planejado
1	Banco criado, dados carregados e permissões verificadas.
2	Terminal, cadastro, Argon2id e autenticação com parâmetros.
3	Ataques de teste, comparação de índices e evidências documentadas.

Entrega e limitações atuais

Exigência da atividade	Material e estado atual
Título, público e problema	Apresentados na abertura.
Identificação do grupo e integrantes	Nomes e matrículas na primeira página.
Canvas de uma página	Página 2, com arquitetura em caixas.
Repositório e README	Repositório criado; README disponível no projeto.
Modelo, campos e proteção	Página 3 e script 01.
Três papéis com DENY	Página 4 e script 02.
Ameaça e prova	Plano e script 03; execução pendente.
Duas consultas e volume	Página 5; scripts 04 e 05.
Base inicial carregada	PENDENTE: scripts disponíveis; carga não comprovada.

Preparação da base

Em uma instância de laboratório SQL Server, executar os scripts 01 e 02 no SSMS com uma identidade de implantação autorizada. O banco dedicado chama-se LoginGuardAcademico. O script 01 interrompe se ele já existir, sem sobrescrever dados. Depois executar o script 03 e guardar a saída.

Os scripts 04 e 05 suportam o experimento de desempenho. O programa de terminal ainda não foi implementado. Em 30/10/2026, a tentativa de conexão à instância local SQLEXPRESS falhou e o serviço foi encontrado parado. Não se afirma que a carga ou os testes passaram.

Repositório e envio

github.com/Guztaver/loginguard-bd
Repositório privado: o professor deverá ter acesso para avaliar os arquivos. Cada participante deve fazer seu próprio envio. O prazo informado na atividade foi 27/10.

Referências técnicas

Microsoft: DENY e exceções de permissão

Microsoft: Dynamic Data Masking e limitações

OWASP: Password Storage Cheat Sheet

OWASP: SQL Injection Prevention Cheat Sheet

Apoio de IA: Codex na preparação da proposta, documentação e scripts iniciais. Todos os integrantes devem revisar e saber explicar cada parte do projeto.

APÊNDICE A

Script de criação e carga

Fonte: sql/01_criacao_carga.sql. Código disponibilizado, ainda sem validação de execução nesta entrega.

```
-- Laboratório fictício. Execute no SSMS com uma identidade de implantação.
-- Se o banco já existir, interrompe sem alterar seu conteúdo.
USE master;
GO
IF DB_ID(N'LoginGuardAcademico') IS NOT NULL
BEGIN
    RAISERROR(N'Banco já existe. Não execute a carga novamente neste banco.', 16, 1);
    SET NOEXEC ON;
END;
GO
CREATE DATABASE LoginGuardAcademico;
GO
USE LoginGuardAcademico;
GO
CREATE TABLE dbo.Conta (
    ContaId int IDENTITY(1,1) NOT NULL CONSTRAINT PK_Conta PRIMARY KEY,
    Nome nvarchar(120) NOT NULL,
    Email varchar(254) MASKED WITH (FUNCTION = 'email()') NOT NULL,
    Bloqueado bit NOT NULL CONSTRAINT DF_Conta_Bloqueado DEFAULT (1),
    CriadoEm datetime2(0) NOT NULL CONSTRAINT DF_Conta_CriadoEm DEFAULT (SYSUTCDATETIME()),
    CONSTRAINT UQ_Conta_Email UNIQUE (Email)
);
CREATE TABLE dbo.Credencial (
    ContaId int NOT NULL CONSTRAINT PK_Credencial PRIMARY KEY,
    HashSenha varchar(512) NULL,
    AtualizadoEm datetime2(0) NOT NULL CONSTRAINT DF_Credencial_Data DEFAULT (SYSUTCDATETIME()),
    CONSTRAINT FK_Credencial_Conta FOREIGN KEY (ContaId) REFERENCES dbo.Conta(ContaId),
    CONSTRAINT CK_Credencial_Formato CHECK (HashSenha IS NULL OR HashSenha LIKE '$argon2id$%')
);
-- O CHECK valida apenas um prefixo. A biblioteca da aplicação deverá validar o hash.
CREATE TABLE dbo.TentativaLogin (
    TentativaId bigint IDENTITY(1,1) NOT NULL CONSTRAINT PK_TentativaLogin PRIMARY KEY,
    ContaId int NULL,
    OcorridoEm datetime2(0) NOT NULL,
    Sucesso bit NOT NULL,
    IpOrigem varchar(45) MASKED WITH (FUNCTION = 'default()') NOT NULL,
    CONSTRAINT FK_TentativaLogin_Conta FOREIGN KEY (ContaId) REFERENCES dbo.Conta(ContaId)
);
GO
SET XACT_ABORT ON;
BEGIN TRANSACTION;
DECLARE @n int = 1;
WHILE @n <= 50
BEGIN
    INSERT dbo.Conta (Nome, Email)
    VALUES (CONCAT(N'Pessoa Fictícia ', @n), CONCAT('pessoa', @n, '@example.invalid'));
    SET @n += 1;
END;
INSERT dbo.Credencial (ContaId, HashSenha)
SELECT ContaId, NULL FROM dbo.Conta;
-- Eventos históricos sintéticos: não correspondem a autenticações reais.
SET @n = 1;
WHILE @n <= 500
BEGIN
    INSERT dbo.TentativaLogin (ContaId, OcorridoEm, Sucesso, IpOrigem)
    VALUES (1 + ((@n - 1) % 50), DATEADD(MINUTE, @n, CONVERT(datetime2(0), '20261001', 112)),
            CASE WHEN @n % 3 = 0 THEN 0 ELSE 1 END,
            CONCAT('192.0.2.', 1 + (@n % 254)));
    SET @n += 1;
END;
COMMIT;
SELECT N'Conta' AS Tabela, COUNT_BIG(*) AS Linhas FROM dbo.Conta
UNION ALL SELECT N'Credencial', COUNT_BIG(*) FROM dbo.Credencial
UNION ALL SELECT N'TentativaLogin', COUNT_BIG(*) FROM dbo.TentativaLogin;
GO
```

APÊNDICE B

Papéis e permissões

Fonte: sql/02_permissoes.sql. Código disponibilizado, ainda sem validação de execução nesta entrega.

```
USE LoginGuardAcademico;
GO
CREATE VIEW dbo.vContas AS
SELECT ContaId, Nome, Email, Bloqueado, CriadoEm FROM dbo.Conta;
GO
CREATE VIEW dbo.vHistorico AS
SELECT TentativaId, ContaId, OcorridoEm, Sucesso FROM dbo.TentativaLogin;
GO
CREATE ROLE lg_administrador AUTHORIZATION dbo;
CREATE ROLE lg_suporte AUTHORIZATION dbo;
CREATE ROLE lg_auditor AUTHORIZATION dbo;
GO
GRANT SELECT ON OBJECT::dbo.vContas TO lg_administrador, lg_suporte, lg_auditor;
GRANT SELECT ON OBJECT::dbo.vHistorico TO lg_administrador, lg_auditor;
GRANT INSERT ON OBJECT::dbo.Conta TO lg_administrador;
-- Permite localizar a conta no WHERE do UPDATE sem expor as demais colunas.
GRANT SELECT ON OBJECT::dbo.Conta (ContaId) TO lg_administrador;
GRANT UPDATE ON OBJECT::dbo.Conta (Nome, Email, Bloqueado) TO lg_administrador;
GRANT INSERT ON OBJECT::dbo.Credencial TO lg_administrador;
GRANT UPDATE ON OBJECT::dbo.Credencial (HashSenha, AtualizadoEm) TO lg_administrador;
DENY SELECT ON OBJECT::dbo.Credencial TO lg_administrador, lg_suporte, lg_auditor;
DENY INSERT, UPDATE, DELETE ON OBJECT::dbo.Conta TO lg_suporte, lg_auditor;
DENY INSERT, UPDATE, DELETE ON OBJECT::dbo.Credencial TO lg_suporte, lg_auditor;
DENY INSERT, UPDATE, DELETE ON OBJECT::dbo.TentativaLogin TO lg_administrador, lg_suporte, lg_auditor;
DENY UNMASK TO lg_administrador, lg_suporte, lg_auditor;
GO
CREATE USER demo_admin WITHOUT LOGIN;
CREATE USER demo_suporte WITHOUT LOGIN;
CREATE USER demo_auditor WITHOUT LOGIN;
ALTER ROLE lg_administrador ADD MEMBER demo_admin;
ALTER ROLE lg_suporte ADD MEMBER demo_suporte;
ALTER ROLE lg_auditor ADD MEMBER demo_auditor;
GO
-- Nenhum destes usuários é db_owner. Não usar sysadmin para demonstrar DENY.
-- SELECT das views funciona por cadeia de propriedade; o mascaramento continua aplicável.
```